บทนำเสนอ MIPS Simulator — Factorial & nCk

Nov 2, 2026 · @KurtSakon

ภาพรวม

พูดรวม demo **ประมาณ 10 นาที** (ยืดหยุ่นได้ 5–10) เล่าเหมือนพาอาจารย์เดินดูโปรแกรมจริง เห็นอะไรบนจอก็อธิบายอันนั้น

แกนของเรื่อง: **command line (`main.py`) คือตัวหลักตามโจทย์ ส่วน GUI เป็นสิ่งที่เราเพิ่มให้ใช้สะดวกขึ้น ใช้ CPU ตัวเดียวกัน**

ช่วง	เวลา	พาไปดูอะไร
1. โครงสร้างโปรเจค	0:00–3:00	tree ไฟล์, workflow, ปรับปรุงจาก V7.1, โค้ด mul + stack, สามเหลี่ยมปาสคาล
2. Command line	3:00–4:30	รัน factorial แบบปกติ, `-q` , `-t`
3. เปิด GUI	4:30–5:00	`--gui` และหน้าตาโดยรวม
4. พาดูปุ่ม + Run factorial	5:00–7:00	แถบเครื่องมือ, editor, registers, memory/stack, แท็บล่าง
5. Overflow 13!	7:00–7:40	คำเตือนสีส้ม
6. nCk + DEC	7:40–8:50	Examples, memory 0x4000, ปุ่ม DEC
7. แก้โค้ดสด + สรุป	8:50–10:00	แก้คำสั่งให้ผิดดู error, .exe, สรุป

เวลาไม่พอให้ตัดช่วง 7 ส่วนแก้โค้ดสด แล้วพูดแค่ประโยคสรุป

ช่วงเปิด: โครงสร้างโปรเจค (ประมาณ 3 นาที)

พูดก่อนเข้า demo ใช้ 4 สไลด์: **โครงสร้างไฟล์ → workflow → สิ่งที่ปรับปรุงจาก V7.1 → โค้ด assembly** (ตามด้วยสไลด์โค้ดที่เลือกไว้)

สไลด์ 1 — โครงสร้างไฟล์

คัดลอกกล่องนี้ไปใส่สไลด์ได้เลย แล้วใส่สีตามสัญลักษณ์ (เช่น ★ เขียว, ✎ ส้ม, ○ เทา)

```
factorial_project/
├─ sim/                          ตัว simulator (Python)
│  ├─ main.py          ✎  จุดเริ่ม: รับคำสั่ง -t -s -q --gui --mem ...
│  ├─ isa.py           ✎  assembler: label 2-pass, ชื่อ register, machine code
│  ├─ cpu.py           ✎  CPU: fetch-decode-execute, JAL/JR, SYSCALL,
overflow
│  ├─ memory.py        ✎  memory: อ่าน/เขียน word 4 byte
│  ├─ gui.py           ★  หน้าต่าง GUI (--gui)
│  ├─ tui.py           ★  หน้าจอเต็มแบบ DOSBox (--tui)
│  └─ instructions.txt ○  โปรแกรมตัวอย่างของอาจารย์
├─ programs/                     โปรแกรม MIPS assembly ที่เราเขียน
│  ├─ factorial.asm    ★  n! + ฟังก์ชันคูณ mul ที่เขียนเอง
│  └─ nck.asm          ★  nCk ด้วยสามเหลี่ยมปาสคาล (บวกอย่างเดียว)
├─ examples/                  ○  ไฟล์ทดสอบของอาจารย์ (ใช้เช็คว่ายังรันได้)
├─ tests/test_all.py          ★  เทสต์อัตโนมัติ 11 ชุด
├─ MIPS Simulator.pyw / .exe  ★  ดับเบิลคลิกเปิด GUI
└─ README.md, FULL_GUIDE.md   ★  คู่มือ

✎ = ปรับปรุงจาก V7.1 ของอาจารย์    ★ = เขียนใหม่ทั้งไฟล์    ○ = ของอาจารย์เดิม
```

ตัวเลขสำหรับพูด: simulator มี Python **6 ไฟล์** (ปรับจาก V7.1 4 ไฟล์ + เขียนใหม่ 2 ไฟล์), assembly **2 ไฟล์**, เทสต์ **1 ไฟล์** ในโค้ดมีป้าย `★ [NEW]` / `★ [MODIFIED]` กำกับทุกจุดที่เพิ่มหรือแก้

สไลด์ 2 — Workflow: ไฟล์ .asm กลายเป็นผลลัพธ์ยังไง

```
factorial.asm
     |  python main.py ../programs/factorial.asm
     ▼
main.py ── อ่านไฟล์ + ตัวเลือก (-t, -q, --gui ...)
     ▼
isa.py ─── รอบ 1: ตัดคอมเมนต์, จด address ของ label
             รอบ 2: แทน label ด้วยตัวเลข → แปลงเป็น machine code 32 bit
     ▼
memory.py — เก็บ machine code เริ่มที่ address 0
     ▼
cpu.py ─── วน fetch → decode → execute → PC+1 จนเจอ SYSCALL 10
     ▼
ผลลัพธ์ ── command line (ตัวหลัก)  หรือ  GUI (--gui, CPU ตัวเดียวกัน)
```

สไลด์ 3 — ปรับปรุงจาก V7.1 อะไรบ้าง

ไฟล์	แก้บั๊ก	เพิ่มใหม่
`isa.py`	**machine code R-type เดิมได้ 00000000** → ตรงเฉลย A01; BEQ/BNE สลับ rs/rt	assembler 2-pass (label), ชื่อ register `$t0` `$sp`, คอมมาและคอมเมนต์
`cpu.py`	SLTI เขียนทับ register, validate พังเมื่อเกิน 32 bit, ไฟล์ไม่มี .ORG error	คำสั่ง 13 ตัว (SUB AND OR SLT SLL JAL JR ...), SYSCALL 1/5/10/11, stack `$sp` = 0x8000, **ตรวจ overflow**, trace/step
`memory.py`	SW/LW เดิมเขียนทั้งค่าลง address เดียว	`read_word` / `write_word` ทีละ 4 byte big-endian
`main.py`	อ่านได้แค่ instructions.txt	เลือกไฟล์ได้ + ตัวเลือก `-t -s -q -v --mem --int --gui --tui`

สไลด์ 4 — โค้ด assembly

ใช้สไลด์โค้ด `mul` และ stack frame ของ factorial แล้วต่อด้วยสามเหลี่ยมปาสคาลของ nCk:

```
      1            แต่ละช่อง = ผลบวก 2 ช่องข้างบน
     1 1           C(i, j) = C(i-1, j-1) + C(i-1, j)
    1 2 1
   1 3 3 1
  1 4 6 4 1        4C2 = 6
```

สไลด์ที่ต้องมี: 1 (tree) และ 3 (ตารางปรับปรุง) ส่วนสไลด์ 2 รวมกับ 1 ได้ถ้าสไลด์เยอะเกิน

บทพูด

> "โปรเจคของพวกผมเอา MIPS simulator V7.1 ของอาจารย์มาปรับปรุงครับ ตัว simulator เป็น Python 6 ไฟล์ มี 4 ไฟล์ที่ปรับจากของอาจารย์คือ main, isa, cpu, memory และเขียนใหม่ 2 ไฟล์คือ gui กับ tui ส่วนโปรแกรม assembly เราเขียน 2 ตัวคือ factorial กับ nCk" *(ชี้ tree)*

> "การทำงานคือ main.py อ่านไฟล์ .asm ส่งให้ isa.py แปลเป็น machine code แบบ 2 รอบ รอบแรกหา label รอบสองแทนเป็นตัวเลข เก็บลง memory แล้ว cpu.py ก็วน fetch-decode-execute จนจบโปรแกรม" *(ชี้ workflow)*

> "สิ่งที่ปรับปรุงมี 2 ส่วนครับ ส่วนแรกคือแก้บั๊ก ที่สำคัญคือ machine code ของคำสั่ง R-type อย่าง ADD เดิมออกมาเป็นศูนย์ทั้งหมด ตอนนี้ตรงกับเฉลยแบบฝึกหัด A01 แล้ว และ SW/LW ที่เดิมเขียนค่าลง address เดียว เราแก้ให้เขียน 4 byte เหมือน MIPS จริง"

> "ส่วนที่สองคือเพิ่มสิ่งที่จำเป็นต่อโจทย์ คือ label จะได้ไม่ต้องนับ offset เอง คำสั่ง JAL/JR กับ stack เพื่อเรียกฟังก์ชัน SYSCALL รับ-พิมพ์ตัวเลข และตรวจ overflow" *(ชี้ตาราง)*

> "ฝั่ง assembly เริ่มจาก factorial ครับ เนื่องจากไม่มีคำสั่งคูณ เราเลยเขียนฟังก์ชัน mul เอง คือบวก a ซ้ำ b ครั้ง แล้ว factorial เรียก mul วนไป ตอนเรียกต้องเก็บ `$ra` ลง stack ก่อน ไม่งั้นจะกลับ main ไม่ได้" *(ชี้สไลด์โค้ด)*

> "ตอนแรกตั้งใจจะเอา factorial มาคำนวณ nCk ด้วยสูตร n! หาร k!(n−k)! แต่พอถึง 13! ค่าเกินที่ register 32 bit เก็บได้ ทั้งที่อาจารย์ต้องการถึง n = 26 เราเลยเปลี่ยนวิธีไปใช้สามเหลี่ยมปาสคาล ทุกช่องเท่ากับผลบวกสองช่องข้างบน ใช้แค่การบวก ไม่มีคูณไม่มีหาร ตัวเลขระหว่างทางไม่เกินคำตอบ เลยได้ถึง n = 26" *(ชี้สามเหลี่ยม)*

> "ต่อไปจะ demo ให้ดูจริงครับ เริ่มจาก command line ก่อน"

บทพูดแบบ demo

ทำ = สิ่งที่กด/พิมพ์ · กล่องคำพูด = พูดตามได้เลย ไม่ต้องท่องเป๊ะ จำแค่ลำดับ พูดก่อนกดทุกครั้ง คนดูจะรู้ว่าต้องมองตรงไหน

1. เปิด — ใช้ "ช่วงเปิด: โครงสร้างโปรเจค" ด้านบนแทน

2. Command line — ตัวหลัก (1 นาที 45 วินาที)

ทำ: สลับไป Command Prompt ที่อยู่ในโฟลเดอร์ `sim` แล้วพิมพ์ `python main.py ../programs/factorial.asm`

> "วิธีใช้หลักคือสั่ง python main.py ตามด้วยไฟล์ assembly ที่จะรัน"

ทำ: ชี้ตารางที่ขึ้นมา

> "ส่วนแรกคือผลของ assembler คอลัมน์ซ้ายคือ address ตรงกลางคือ machine code 32 bit ขวาคือคำสั่ง จะเห็นว่า label อย่าง factorial ถูกแปลงเป็นตัวเลข เช่น JAL 0x30 แล้วโปรแกรมก็ถาม input #1 คือค่า n"

ทำ: พิมพ์ `5` Enter

> "ได้ 120 คือ 5! ข้างล่างเป็น Last STATE คือค่า register 32 ตัวและ memory ตอนจบ บรรทัดสุดท้ายบอกว่ารันไป 101 คำสั่ง"

ทำ: พิมพ์ `python main.py ../programs/factorial.asm -q` ใส่ `5`

> "ถ้าใส่ -q จะเหลือแค่คำตอบ 120 อย่างเดียว เหมาะกับระบบตรวจอัตโนมัติ"

ทำ: พิมพ์ `python main.py ../programs/factorial.asm -t` ใส่ `3` แล้วชี้บรรทัด `line 40 [0000000C] JAL 0x30`

> "-t คือ trace แสดงทุกคำสั่งที่ CPU รันจริง ตัวเลข line คือบรรทัดในไฟล์ .asm ตามด้วย address คำสั่ง และ register ที่เปลี่ยนจากค่าเก่าไปค่าใหม่ เช่นบรรทัด 40 JAL ทำให้ `$ra` กลายเป็น 0x10 คือที่ที่จะกลับมา"

3. เปิด GUI (30 วินาที)

> "แต่การอ่าน trace ยาว ๆ ใน command line ดูยาก เราเลยเพิ่ม --gui ใช้ไฟล์เดิมและ CPU ตัวเดิม แค่เปลี่ยนวิธีแสดงผล"

ทำ: พิมพ์ `python main.py ../programs/factorial.asm --gui`

> "หน้าต่างแบ่งเป็น 4 ส่วนครับ ด้านบนเป็นแถบเครื่องมือ ซ้ายเป็นโค้ด ขวาเป็น register กับ memory ด้านล่างเป็นแท็บผลลัพธ์ และล่างสุดเป็นแถบสถานะที่บอก PC กับจำนวนคำสั่งที่รัน"

4. พาดูทุกส่วน แล้วรัน factorial (2 นาที 30 วินาที)

ทำ: ชี้แถบเครื่องมือจากซ้ายไปขวา

> "ปุ่ม Open ใช้โหลดไฟล์ assembly จากเครื่องได้เลย อาจารย์เอาไฟล์ของอาจารย์มาเปิดก็ได้ Save บันทึกโค้ดที่แก้ Examples เป็นทางลัดเปิดโปรแกรมตัวอย่างทั้งของเราและของอาจารย์"

> "กลุ่มถัดมาคือปุ่มควบคุม CPU: Assemble แปลโค้ดใหม่, Step หรือ F10 รันทีละคำสั่ง, Run หรือ F5 รันต่อเนื่อง ระหว่างรันปุ่มจะเปลี่ยนเป็น Pause, และ Reset เริ่มใหม่"

> "แถบ Speed ปรับความเร็วตั้งแต่ 1 คำสั่งต่อวินาทีจนถึง max สุดท้าย Numbers HEX/DEC สลับตัวเลขทั้งหน้าจอระหว่างฐาน 16 กับฐานสิบ เดี๋ยวจะโชว์ตอน nCk"

ทำ: ชี้ editor ด้านซ้าย

> "ตรงนี้เป็น editor แก้โค้ดในโปรแกรมได้เลย ไม่ต้องเปิดโปรแกรมอื่น แถบสีฟ้าคือคำสั่งที่ CPU จะรันถัดไป คอมเมนต์เป็นสีเทา และคลิกที่เลขบรรทัดเพื่อตั้ง breakpoint ได้ จะขึ้นจุดแดง ตอน Run จะหยุดตรงนั้น"

ทำ: คลิกเลขบรรทัด 69 (`SW $ra, 12($sp)`) ให้เกิดจุดแดง แล้วชี้ฝั่งขวา

> "ฝั่งขวาบนคือ register ครบ 32 ตัวพร้อมชื่อ ตัวไหนเพิ่งเปลี่ยนจะเป็นสีเขียว ข้างล่างคือ memory ตอนนี้ติ๊ก Follow `$sp` ไว้ มันจะตาม stack ให้เอง"

ทำ: ตั้ง Speed ประมาณ 5 → กด **Run** → หน้าต่างเด้งถาม input ใส่ `5` → OK

> "พอโปรแกรมถึง SYSCALL 5 จะมีหน้าต่างเด้งมาถาม บอกว่าเป็น input ครั้งที่เท่าไหร่และมาจากบรรทัดไหน"

ทำ: โปรแกรมหยุดที่ breakpoint กด **Step** 4 ครั้ง ชี้ memory ที่เปลี่ยน

> "ตอนนี้หยุดที่ breakpoint คือต้นฟังก์ชัน factorial จะเห็นว่า `$sp` ลดจาก 8000 เป็น 7FF0 คือจอง stack 16 byte พอกด Step คำสั่ง SW ก็เก็บ `$ra` กับ `$s0` – `$s2` ลง memory เพราะตอนเรียก mul ด้วย JAL ค่า `$ra` จะถูกเขียนทับ"

ทำ: กด **Run** ต่อจนจบ แล้วชี้แท็บด้านล่างทีละแท็บ

> "แท็บ Console มี input #1: 5 สีฟ้าคือค่าที่เราใส่ และ 120 คือสิ่งที่โปรแกรม MIPS พิมพ์เอง"

> "แท็บ Trace คือสิ่งเดียวกับ -t เมื่อกี้ กด Open in large window ดูเต็มจอได้ และแท็บ Machine code คือตาราง address, machine code, เลขบรรทัด และ label"

5. Overflow 13! (45 วินาที)

ทำ: คลิกจุดแดงออก → **Reset** → Speed max → **Run** → ใส่ `13`

> "คราวนี้ลอง 13! ค่าจริงคือ 6,227,020,800 แต่ register 32 bit เก็บได้สูงสุดประมาณ 2,147 ล้าน"

> "เห็นข้อความสีส้ม OVERFLOW บอก address 0x98 คือคำสั่ง ADD ใน mul พร้อมค่าที่ควรได้ ผลสุดท้ายจึงผิด นี่คือข้อจำกัดของขนาด register ไม่ใช่บั๊ก และเป็นเหตุผลที่ nCk ต้องเปลี่ยนวิธี"

6. nCk และปุ่ม DEC (1 นาที 15 วินาที)

ทำ: Examples → `programs/nck.asm`

> "nCk ถ้าใช้สูตร factorial จะล้นตั้งแต่ 13! เราเลยเปลี่ยนไปใช้สามเหลี่ยมปาสคาล ทุกช่องเท่ากับผลบวกสองช่องข้างบน ใช้แค่การบวก เก็บแถวไว้เป็น array ใน memory ที่ 0x4000"

ทำ: เอาติ๊ก Follow `$sp` ออก → ช่อง address พิมพ์ `0x4000` → Enter

> "ตรงนี้คือ memory ที่ 0x4000 ตอนนี้เป็นฐาน 16 อ่านยาก เราเลยทำปุ่ม DEC ไว้ กดทีเดียวเปลี่ยนเป็นฐานสิบทั้งหน้าจอ ทั้ง register, memory และ trace"

ทำ: กด **DEC** → Speed 8 → **Run** → ใส่ `10 4` → OK (รอประมาณ 7 วินาที)

> "เห็นตัวเลข 1, 10, 45, 120, 210 ค่อย ๆ ถูกเขียนลง memory นี่คือแถวที่ 10 ของสามเหลี่ยมปาสคาล คำตอบ 10C4 = 210 อยู่ช่องสุดท้ายและขึ้นใน Console วิธีนี้ใช้ได้ถึง n = 26 ตามที่อาจารย์กำหนด"

7. แก้โค้ดสด และสรุป (1 นาที)

ทำ: ใน editor แก้ `ADD` บรรทัดใดก็ได้เป็น `ADDX` → กด **Assemble**

> "ถ้าเขียนผิด บรรทัดนั้นขึ้นแดงและ Console บอกเลขบรรทัดกับสาเหตุ อาจารย์จะลองแก้โค้ดหรือเขียนโปรแกรมใหม่ในนี้เลยก็ได้"

ทำ: แก้กลับเป็น `ADD` (หรือ Ctrl+Z) → ปิดหน้าต่าง **ไม่ต้อง Save**

> "สรุปคือ ตัวหลักของงานคือ main.py ที่รันจาก command line ส่วน GUI เป็นตัวเสริมที่ใช้ CPU ตัวเดียวกัน เราเขียน factorial ด้วยฟังก์ชันคูณของเราเองถูกถึง 12! เจอข้อจำกัด 32 bit ที่ 13! และเปลี่ยนวิธีให้ nCk ได้ถึง n = 26"

> "ถ้าอาจารย์อยากลองเอง เปิด MIPS Simulator.exe ได้เลยโดยไม่ต้องติดตั้ง Python ส่วนการตรวจแบบ autolab ใช้ python main.py ตามปกติ ขอบคุณครับ"

แผนสำรอง

ปัญหา	ทำทันที
GUI ไม่เปิดหรือค้าง	อยู่ใน command line ต่อ: `-t` ดู trace, `python main.py ../programs/nck.asm --mem 0x4000` ดู array
เครื่องห้องไม่มี Python	เปิด `MIPS Simulator.exe` หรือใช้โน้ตบุ๊กตัวเอง
`No such file`	เช็คว่าอยู่ในโฟลเดอร์ `sim` และใช้ `../programs/`
หน้าต่าง input ค้าง	กด Stop program → Reset → Run
เวลาไม่พอ	ตัดช่วง 7 และตัด `-q` ในช่วง 2

เตรียมคำสั่งทั้งหมดไว้ใน Notepad แล้ว copy มาวาง และอัดคลิปตอนซ้อมไว้ 1 คลิป (Win + G) เผื่อทุกอย่างพัง

คำถามที่น่าจะโดนถาม

ตอบสั้น ๆ ก่อน 1–2 ประโยค ถ้าอาจารย์ถามต่อค่อยลงรายละเอียด ถ้าไม่รู้ให้พูดตรง ๆ ว่า "ส่วนนั้นเรายังไม่ได้ทำ/ยังไม่แน่ใจครับ" ดีกว่าเดา

เรื่องโจทย์และการคูณ

- **ฟังก์ชันคูณที่เขียนเองอยู่ตรงไหน?** — ฟังก์ชัน `mul` ท้ายไฟล์ factorial.asm บวก a ซ้ำ b ครั้ง ไม่มี MUL เลยทั้งโปรแกรม
- **ทำไมไม่เพิ่มคำสั่ง MUL ให้ simulator ไปเลย?** — โจทย์ให้สมมติว่าไม่มีคำสั่งคูณ เหมือน CPU เล็ก ๆ ที่ไม่มีวงจรคูณ ต้องทำด้วย software
- **บวกซ้ำช้าไหม?** — ช้า จำนวนรอบเท่ากับตัวคูณ วิธีที่เร็วกว่าคือ shift-and-add (เลื่อน bit แล้วบวก วนไม่เกิน 32 รอบ) เราเลือกบวกซ้ำเพราะตรงกับ `factorial_no_mul_1.py` ของอาจารย์และอ่านง่าย

- **ทำไมได้แค่ 12!?** — 13! = 6,227,020,800 เกิน 2,147,483,647 ที่ register 32 bit แบบมีเครื่องหมายเก็บได้ เป็นข้อจำกัดของฮาร์ดแวร์ ถ้าจะได้มากกว่านี้ต้องใช้ 2 register เก็บ 64 bit
- **MIPS จริงทำอะไรตอน overflow?** — ADD/ADDI/SUB เกิด exception หยุดโปรแกรม ส่วน ADDU ไม่ตรวจ simulator เราเลือกเตือนแล้วรันต่อ เพื่อให้เห็นว่าผลผิดไปยังไง

เรื่อง nCk

- **nCk ใช้ factorial ไหม?** — ไม่ใช้ เป็นโปรแกรมแยกที่เปลี่ยนวิธี เพราะสูตร factorial ล้นตั้งแต่ 13!
- **ทำไม array อยู่ที่ 0x4000?** — เป็นที่ว่างตรงกลาง ไม่ชนโปรแกรมที่เริ่มที่ 0 และไม่ชน stack ที่เริ่ม 0x8000 แล้วโตลง
- **ทำไมวนขวาไปซ้าย?** — ใช้ array แถวเดียวอัปเดตทับกัน ถ้าซ้ายไปขวา row[j−1] จะถูกเปลี่ยนก่อนที่ row[j] จะใช้
- **ได้ถึง n เท่าไหร่?** — ทดสอบครบทุกคู่ถึง 26 (378 คู่) ใช้ได้จริงถึง 33 เพราะ 34C17 เกิน 32 bit
- **SLL คือการคูณไม่ใช่เหรอ?** — เป็นการเลื่อน bit ไม่ใช่คำสั่งคูณ คูณได้แค่ด้วยกำลังของ 2 ใช้หา address ของ array (j × 4)

เรื่อง simulator

- **แก้อะไรจากของอาจารย์บ้าง?** — บั๊ก 7 จุด เช่น machine code R-type, BEQ/BNE สลับ rs/rt, SW/LW 4 byte, SLTI เขียนทับ register ต้นทาง ทุกจุดมีป้าย `★ [MODIFIED]` พร้อมเหตุผลในโค้ด
- **assembler แปล label ยังไง?** — 2 รอบ รอบแรกจด address ของทุก label รอบสองแทนค่า BEQ/BNE ใช้ offset = (ปลายทาง − คำสั่งถัดไป) ÷ 4 ส่วน J/JAL ใช้ address ÷ 4
- **รู้ได้ยังไงว่า machine code ถูก?** — เทสต์อัตโนมัติเทียบ 13 คำสั่งกับเฉลย A01 พร้อมแกะ bit ให้ดูสดได้ เช่น `ADD $s2,$s1,$zero` = opcode 000000, rs 10001, rt 00000, rd 10010, shamt 00000, funct 100000
- **PC เก็บอะไร?** — เก็บเป็น address ÷ 4 ตามแบบ V7.1 เวลาแสดงผลคูณ 4 กลับเป็น byte address
- **GUI กับ command line ต่างกันไหม?** — CPU ตัวเดียวกัน (`cpu.py`) GUI แค่เรียกทีละคำสั่งแล้ววาดจอ
- **จำลอง pipeline ไหม?** — ไม่ เป็นแบบ single-cycle ทำทีละคำสั่งจบแล้วค่อยทำคำสั่งถัดไป ไม่มี delay slot
- **ใครทำส่วนไหน / ใช้ AI ไหม?** — ตอบตามความจริง และให้แต่ละคนอธิบายส่วนที่ตัวเองรับผิดชอบได้เอง อาจารย์มักทดสอบด้วยการให้อธิบายบรรทัดใดบรรทัดหนึ่ง

ถ้าอาจารย์ให้แก้สด

- "ลองใส่ 0 หรือ 1" → ได้ 1 (loop ไม่ทำงาน)
- "ลอง k มากกว่า n" → ได้ 0
- "เปิดไฟล์ของอาจารย์" → `python main.py` (ได้ `$3` = 0x37 = 55) หรือ GUI → Open
- "แก้คำสั่งใน GUI ให้ผิด" → บรรทัดขึ้นแดงพร้อม error บอกเลขบรรทัด

Checklist และวิธีซ้อม

ก่อนวันนำเสนอ

- [ ] รัน demo ทุกขั้นบนเครื่องที่จะใช้จริง (Windows) ครบ 1 รอบ
- [ ] รันเทสต์ `python -m unittest discover -s tests -v` ต้องขึ้น OK
- [ ] รัน `build_exe.bat` ใหม่ แล้วลองเปิด .exe
- [ ] แก้สไลด์: เปลี่ยน "ต่อยอด" เป็น "เปลี่ยนวิธี" และเพิ่ม `--gui`, HEX/DEC
- [ ] เตรียมคำสั่ง demo ไว้ใน Notepad
- [ ] ถ่ายคลิปสำรอง demo 1 คลิป
- [ ] ซ้อมพูดจับเวลาอย่างน้อย 2 รอบ

15 นาทีก่อนขึ้น

- [ ] เปิด Command Prompt ที่ `cd` เข้า `sim` แล้ว ขยายตัวอักษร
- [ ] เปิดสไลด์ + ไฟล์ factorial.asm ไว้เทียบ
- [ ] ปิดแจ้งเตือน (LINE, Discord) และเสียบสายชาร์จ
- [ ] ลอง `python main.py ../programs/factorial.asm -q` 1 รอบให้แน่ใจว่าเครื่องพร้อม

วิธีซ้อมให้ได้ผล

1. **รอบแรก** อ่านบทออกเสียงคนเดียว จับเวลา ถ้าเกิน 10 นาทีให้ตัดช่วง 7
2. **รอบสอง** ซ้อมพร้อม demo จริง ให้เพื่อนในกลุ่มฟัง แล้วให้เพื่อนสุ่มถาม 3 ข้อจากส่วนคำถาม
3. ทุกคนในกลุ่มต้องตอบได้ 3 ข้อนี้ด้วยตัวเอง:
   - mul ทำงานยังไง
   - ทำไมต้องเก็บ `$ra` ลง stack
   - ทำไม 13! ผิด และทำไม nCk ต้องเปลี่ยนวิธี

รายละเอียดทุกคำสั่งอยู่ใน `FULL_GUIDE.md` ส่วนที่ 18 ของโปรเจค